

Morphic Studio Software Architecture Specification

Document 40 — Backend Architecture Specification

Version: 1.0

Status: Approved

1. Purpose

This document defines the backend architecture of Morphic Studio.

It specifies how business capabilities, APIs, AI orchestration, authentication, storage, background processing, events, and integrations are structured to provide a scalable, secure, and maintainable platform.

The backend is responsible for enforcing business rules while exposing a stable contract to the frontend and external integrations.

2. Backend Philosophy

The backend follows eight core principles:

2.1 Business Capability Architecture

The backend is organized around business capabilities rather than technical layers.

Examples:

- Story
- Characters
- Worlds
- Assets
- Production
- Publishing

- AI
- Users
- Authentication
- Billing (future)

Each capability owns:

- API endpoints
 - Business rules
 - Validation
 - Persistence
 - Events
 - Background jobs
-

2.2 Domain Ownership

Each domain owns its own business logic.

No capability should directly manipulate another capability's internal data.

Communication occurs through well-defined interfaces or events.

2.3 API-First Design

Every backend capability exposes stable, documented APIs.

Internal implementation may evolve without breaking API contracts.

2.4 Event-Driven Communication

Long-running and cross-domain operations communicate using domain events.

Examples:

ProjectCreated

StoryBibleGenerated

CharacterUpdated

AssetGenerated

RenderCompleted

ExportFinished

NotificationCreated

This reduces coupling between services.

2.5 Asynchronous by Default

Operations expected to take more than a few seconds should execute as background jobs.

Examples:

AI generation

Rendering

Asset indexing

Search indexing

Exports

Analytics aggregation

2.6 Stateless Services

Application services remain stateless whenever practical.

Persistent state belongs in databases, caches, or object storage.

2.7 Security by Design

Authentication, authorization, auditing, and validation are enforced consistently across all capabilities.

2.8 Observability

Every important action produces structured logs, metrics, traces, and audit events while respecting user privacy.

3. Technology Stack

Language:

TypeScript

Framework:

NestJS

Database:

PostgreSQL

ORM:

Prisma

Cache:

Redis

Queue:

BullMQ

Search:

Meilisearch

Object Storage:

S3-compatible storage

Authentication:

Better Auth (or equivalent)

Realtime:

WebSockets

Message/Event Bus:

Abstract interface supporting future replacement

4. Capability Structure

Each capability follows the same internal organization.

Example:

story/

controllers/

services/

repositories/

entities/

dto/

validators/

events/

jobs/

tests/

documentation/

This consistency improves developer productivity and AI-assisted coding.

5. API Architecture

Expose versioned APIs.

Examples:

/api/v1/projects

/api/v1/story

/api/v1/story-bible

/api/v1/characters

/api/v1/worlds

/api/v1/assets

/api/v1/production

/api/v1/export

/api/v1/users

Future breaking changes should be introduced through new API versions.

6. Authentication & Authorization

Authentication responsibilities:

User identity

Sessions

Tokens

Multi-device support

Password reset

Email verification

OAuth providers (future)

Authorization responsibilities:

Role-based permissions

Project ownership

Team permissions

Administrative access

Future organization support

7. Unified AI Gateway

All AI requests must pass through a centralized AI Gateway.

Responsibilities:

Provider selection

Prompt assembly

Model routing

Rate limiting

Retries

Cost tracking

Fallback providers

Response normalization

Logging

The rest of the application must never depend directly on individual AI vendors.

8. Background Processing

Background workers handle:

AI generation

Rendering

Asset optimization

Thumbnail generation

Search indexing

Export generation

Analytics aggregation

Notifications

Workers should report progress for long-running tasks.

9. Event System

Every important business action emits domain events.

Examples:

ProjectCreated

StoryUpdated

CharacterDeleted

WorldModified

AssetImported

RenderStarted

RenderCompleted

ExportSucceeded

Events must be versioned and documented.

10. Storage Strategy

Use the appropriate storage for each type of data:

PostgreSQL

Structured relational data

Redis

Caching

Queues

Temporary state

S3-Compatible Storage

Images

Videos

Audio

Documents

Generated assets

Meilisearch

Full-text search

Asset discovery

Story search

11. Security Standards

Implement:

Input validation

Output sanitization

Rate limiting

CSRF protection

XSS protection

SQL injection prevention

Encryption in transit

Encryption at rest

Audit logging

Secret management

Least-privilege access

12. Scalability Strategy

Design for horizontal scalability.

Application servers remain stateless.

Background workers scale independently.

Object storage scales independently.

Caching reduces database load.

The architecture should support increasing workloads without redesign.

13. Error Handling

Standardize backend responses.

Include:

Error code

Human-readable message

Developer details (when appropriate)

Trace identifier

Suggested recovery action

Never expose sensitive implementation details.

14. Observability

Monitor:

API latency

Queue health

Database performance

Cache utilization

AI provider latency

Error rates

Storage utilization

Search indexing

Render queue

System health

Support dashboards, alerts, and distributed tracing.

15. Acceptance Criteria

The backend architecture is successful if:

Capabilities remain independent.

Business rules are isolated.

AI providers are replaceable.

Background work does not block user interactions.

APIs remain stable.

Security is consistently enforced.

The system scales without major architectural changes.

Developers can extend capabilities without affecting unrelated modules.

Conclusion

The Morpheic Studio backend architecture establishes a secure, scalable, event-driven foundation for an AI-native creative platform. By organizing the system around business capabilities, enforcing clean API contracts, centralizing AI orchestration, and embracing asynchronous processing, the backend is prepared to support both Version 1 and long-term platform growth.